

Tentu, ini adalah penjelasan mendalam mengenai kode C# tersebut. Kode ini berfungsi sebagai **kalkulator histogram** untuk sebuah gambar digital.

Secara sederhana, kode ini membaca setiap titik (piksel) pada gambar, melihat seberapa terang warnanya (0-255), dan menghitung berapa banyak titik yang memiliki kecerahan tersebut.

Berikut adalah penjelasan per bagian dan per baris kodenya:

1. Struktur Dasar dan Definisi

C#

```
using System.Drawing.Drawing2D;

namespace MiniPhotoshop.Logic.Histogram
{
    public static class HistogramCalculator
    {
```

- **using System.Drawing.Drawing2D;**: Mengimpor *library* grafis tingkat lanjut. Meskipun di kode ini yang paling krusial sebenarnya adalah System.Drawing (untuk kelas Bitmap dan Color), baris ini menyiapkan konteks grafis.
- **namespace MiniPhotoshop.Logic.Histogram**: Ini adalah alamat atau "folder logis" untuk kode ini agar tertata rapi dalam proyek MiniPhotoshop.
- **public static class HistogramCalculator**: Mendefinisikan kelas bernama HistogramCalculator.
 - **static**: Berarti kita tidak perlu membuat objek baru (tidak perlu new HistogramCalculator()) untuk menggunakan fungsi di dalamnya. Kita bisa memanggilnya langsung.

2. Fungsi Utama: Calculate

C#

```
// Menghitung array histogram dari Bitmap untuk channel tertentu
public static int[] Calculate(Bitmap bmp, int channel)
{
    int[] hist = new int[256];
```

- **public static int[] Calculate(...):** Ini adalah metode utama.
 - Input: Bitmap bmp (gambaranya) dan int channel (kode warna yang mau dihitung: 0=Merah, 1=Hijau, dll).
 - Output: int[] (sebuah array berisi angka-angka integer).
- **int[] hist = new int[256];**: Membuat wadah kosong (array) dengan 256 slot (indeks 0 sampai 255).
 - **Mengapa 256?** Karena dalam format gambar digital standar (8-bit), intensitas warna berkisar dari 0 (hitam/gelap total) hingga 255 (putih/terang total).
 - Slot ke-0 akan menyimpan jumlah piksel bernilai 0. Slot ke-255 menyimpan jumlah piksel bernilai 255.

3. Loop (Perulangan) Membaca Piksel

C#

```
for (int y = 0; y < bmp.Height; y++)
{
    for (int x = 0; x < bmp.Width; x++)
    {
        Color pixelColor = bmp.GetPixel(x, y);
        int val = 0;
```

- **for (int y ...)** dan **for (int x ...)**: Ini adalah *nested loop* (perulangan bersarang). Kode ini akan menelusuri gambar baris demi baris, dari kiri atas ke kanan bawah.
- **Color pixelColor = bmp.GetPixel(x, y);**: Pada koordinat (x, y), komputer mengambil data warnanya dan menyimpannya di variabel pixelColor. Data ini berisi komponen R (Red), G (Green), B (Blue), dan A (Alpha).
- **int val = 0;**: Menyiapkan variabel sementara untuk menyimpan nilai intensitas yang akan kita hitung.

4. Logika Pemilihan Channel (Switch Case)

Bagian ini menentukan data apa yang akan diambil dari piksel tersebut.

C#

```
switch (channel)
{
    case 0: // Red
        val = pixelColor.R;
        break;
    case 1: // Green
        val = pixelColor.G;
        break;
    case 2: // Blue
        val = pixelColor.B;
        break;
```

- Jika channel adalah **0**, ambil nilai merah (pixelColor.R).
- Jika channel adalah **1**, ambil nilai hijau (pixelColor.G).
- Jika channel adalah **2**, ambil nilai biru (pixelColor.B).

Kasus Khusus: Grayscale

C#

```
case 3: // Grayscale (Penting: gunakan rumus Grayscale)
    val = (int)(0.3 * pixelColor.R + 0.59 * pixelColor.G + 0.11 * pixelColor.B);
    break;
```

- Ini adalah metode mengubah gambar berwarna menjadi hitam putih. Kita tidak bisa sekadar merata-rata $(R+G+B)/3$ karena mata manusia memiliki sensitivitas berbeda terhadap warna.
- Rumus Luminosity: Mata manusia paling sensitif terhadap Hijau, lalu Merah, dan paling sedikit Biru. Oleh karena itu bobotnya:

$$\text{Grayscale} = 0.3R + 0.59G + 0.11B$$

- Hasil perhitungan (yang berupa desimal/float) di-*casting* (int) agar menjadi bilangan bulat.

C#

```
default:
    val = pixelColor.R;
    break;
}
```

- **default:** Jika input channel bukan 0, 1, 2, atau 3, secara otomatis kode akan mengambil warna Merah (sebagai pencegahan error).

5. Pengisian Data Histogram

C#

```
// Pastikan val berada dalam batas [0, 255]
if (val >= 0 && val <= 255)
{
    hist[val]++;
}
}
return hist;
}
```

- **if (val >= 0 && val <= 255):** Pengecekan keamanan (*sanity check*). Meskipun `GetPixel` biasanya selalu mengembalikan 0-255, ini mencegah error *IndexOutOfRangeException* jika terjadi anomali perhitungan.
 - **hist[val]++:** Ini adalah inti dari algoritma histogram.
 - Misalnya, piksel saat ini memiliki nilai merah 100.
 - Maka, kita pergi ke array `hist` di nomor urut 100, lalu menambahkan isinya dengan 1.
 - Jika nanti ketemu piksel bernilai 100 lagi, isinya ditambah lagi.
 - Hasil akhirnya: `hist[100]` akan berisi **total jumlah piksel** yang memiliki nilai 100.
 - **return hist:** Mengembalikan array yang sudah terisi penuh hitungan frekuensi ke pemanggil fungsi.
-

Ringkasan Alur Kerja

1. Siapkan keranjang kosong berjumlah 256 buah (nomor 0-255).
2. Ambil satu per satu titik pada gambar.
3. Cek nilai warnanya (misal: Merah = 50).
4. Masukkan satu batu ke keranjang nomor 50.
5. Ulangi sampai semua titik gambar habis.
6. Serahkan keranjang-keranjang tersebut sebagai hasil.

Catatan Kinerja (Insight Tambahan)

Kode ini menggunakan `bmp.GetPixel(x, y)`. Untuk gambar kecil, ini tidak masalah. Namun, untuk gambar resolusi tinggi (misal 12 Megapiksel), metode ini akan sangat lambat karena proses penguncian memori dilakukan berulang-ulang per piksel.

Dalam aplikasi *real-world* performa tinggi, biasanya programmer menggunakan metode **LockBits** dan **Marshal.Copy** (pointer arithmetic) untuk membaca data piksel sekaligus dalam memori, yang bisa 100x lebih cepat daripada loop `GetPixel` ini.

Apakah Anda ingin saya tunjukkan bagaimana cara mengoptimalkan kode ini agar lebih cepat menggunakan `LockBits`?